



VLSI Lab

CSE 405

Experiment Number: 02

Design and Simulation of a 1-Bit Full Adder Using Verilog HDL in Xilinx ISE

Date: 27/09/2026

Submitted by:

Name : Md. Seeman Islam Seemanto

ID : 22201095

Section : C1

Semester : 4.2

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

University of Asia Pacific

Experiment Name

Design and Simulation of a 1-Bit Full Adder Using Verilog HDL

Objective

1. Get a clear picture of the Boolean logic and working principle behind a 1-bit full adder.
2. Model a 1-bit full adder in Verilog HDL using dataflow (assign) statements.
3. Produce the RTL schematic of the design inside Xilinx ISE.
4. Build a Verilog testbench to functionally verify the design.
5. Run the design through the ISim simulator and observe the Sum and Cout waveforms.
6. Work out the expected Sum and Cout by hand for all 8 possible input combinations and check them against the truth table.
7. Compare the simulated results to the hand-derived truth table to confirm the circuit behaves correctly.
8. Keep the source files and experiment record in a GitHub repository.

Hardware and Software Requirements

Tool / Component	Details
Xilinx ISE 14.7	The environment used to design, synthesize and verify the 1-bit full adder in Verilog.
Verilog HDL	The hardware description language used to model the full adder through dataflow (assign) statements.
ISim Simulator	Used for the behavioral simulation that checks the full adder's functionality.
Artix-7 / Spartan-6 Device	The target FPGA family chosen inside Xilinx ISE for synthesis and implementation.
Computer / Virtual Machine	A Windows 7 virtual machine running inside VirtualBox, used to host Xilinx ISE 14.7 for the design and simulation work.

Full Adder

A full adder is a combinational circuit that adds three single-bit binary values together: two operand bits, A and B, plus an incoming carry bit, Cin, supplied by the previous, less significant stage. It generates two results — a Sum bit and an outgoing carry bit, Cout.

A half adder, by contrast, only has room for two operands (A and B) and has no way to absorb a carry from an earlier stage. That limitation rules it out for multi-bit addition, because in an n-bit adder every position past the least significant bit needs to fold in whatever carry arrived from its neighbour on the right. Adding the extra Cin input is what lets a full adder solve this: many full adders can be linked in a ripple-carry chain to build an adder of any width, which is why the full adder is treated as the basic building block behind adders, subtractors and ALUs alike.

Functionally, the Sum line goes HIGH whenever an odd number of the three inputs are HIGH — the same behaviour an XOR gate produces — while Cout goes HIGH whenever two or more of the three inputs are HIGH, i.e. whenever a carry actually needs to be generated and forwarded.

Boolean Expression Derivation

Starting from that definition, the Sum and Carry-out equations for a full adder work out to:

$$\begin{aligned}\text{SUM} &= A \oplus B \oplus \text{Cin} \\ \text{Cout} &= A.B + \text{Cin}(A \oplus B) = A.B + B.\text{Cin} + A.\text{Cin}\end{aligned}$$

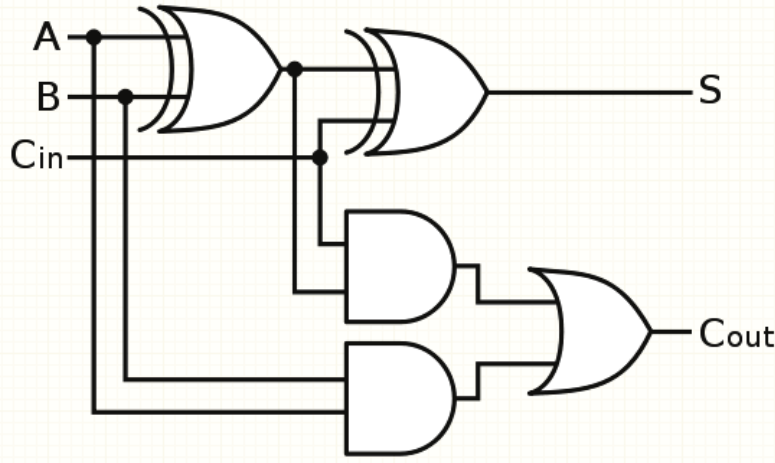


Fig : Full Adder Logic Circuit (Gate level)

In these equations $\oplus$ is XOR, "." is AND and "+" is OR. Sum is written as a 3-input XOR because a binary sum bit is 1 exactly when an odd count of the operands is 1. Cout follows from noticing that a carry has to appear whenever at least two of the three inputs (A, B, Cin) are 1 — captured here as three AND terms combined through an OR.

Manual Calculation for All 8 Input Combinations

Applying the two Boolean expressions above, the Sum and Cout values were worked out by hand for every one of the eight possible A, B, Cin combinations, as shown next. These hand-worked values are exactly what the truth table and the simulation results are checked against later.

Case 1: A = 0, B = 0, Cin = 0

$$\text{Sum} = A \oplus B \oplus \text{Cin} = (0 \oplus 0) \oplus 0 = 0 \oplus 0 = 0$$

$$\text{Cout} = A.B + B.\text{Cin} + A.\text{Cin} = (0.0) + (0.0) + (0.0) = 0 + 0 + 0 = 0$$

Case 2: A = 0, B = 0, Cin = 1

$$\text{Sum} = A \oplus B \oplus \text{Cin} = (0 \oplus 0) \oplus 1 = 0 \oplus 1 = 1$$

$$\text{Cout} = A.B + B.\text{Cin} + A.\text{Cin} = (0.0) + (0.1) + (0.1) = 0 + 0 + 0 = 0$$

Case 3: A = 0, B = 1, Cin = 0

$$\text{Sum} = A \oplus B \oplus \text{Cin} = (0 \oplus 1) \oplus 0 = 1 \oplus 0 = 1$$

$$\text{Cout} = A.B + B.\text{Cin} + A.\text{Cin} = (0.1) + (1.0) + (0.0) = 0 + 0 + 0 = 0$$

Case 4: A = 0, B = 1, Cin = 1

$$\text{Sum} = A \oplus B \oplus \text{Cin} = (0 \oplus 1) \oplus 1 = 1 \oplus 1 = 0$$

$$\text{Cout} = A.B + B.\text{Cin} + A.\text{Cin} = (0.1) + (1.1) + (0.1) = 0 + 1 + 0 = 1$$

Case 5: A = 1, B = 0, Cin = 0

$$\text{Sum} = A \oplus B \oplus \text{Cin} = (1 \oplus 0) \oplus 0 = 1 \oplus 0 = 1$$

$$\text{Cout} = A.B + B.\text{Cin} + A.\text{Cin} = (1.0) + (0.0) + (1.0) = 0 + 0 + 0 = 0$$

Case 6: A = 1, B = 0, Cin = 1

$$\text{Sum} = A \oplus B \oplus \text{Cin} = (1 \oplus 0) \oplus 1 = 1 \oplus 1 = 0$$

$$\text{Cout} = A.B + B.\text{Cin} + A.\text{Cin} = (1.0) + (0.1) + (1.1) = 0 + 0 + 1 = 1$$

Case 7: A = 1, B = 1, Cin = 0

$$\text{Sum} = A \oplus B \oplus \text{Cin} = (1 \oplus 1) \oplus 0 = 0 \oplus 0 = 0$$

$$\text{Cout} = A.B + B.\text{Cin} + A.\text{Cin} = (1.1) + (1.0) + (1.0) = 1 + 0 + 0 = 1$$

Case 8: A = 1, B = 1, Cin = 1

$$\text{Sum} = A \oplus B \oplus \text{Cin} = (1 \oplus 1) \oplus 1 = 0 \oplus 1 = 1$$

$$\text{Cout} = A.B + B.\text{Cin} + A.\text{Cin} = (1.1) + (1.1) + (1.1) = 1 + 1 + 1 = 1$$

Port Define:

Port Name	Direction	Bus	MSB	LSB
A	in	☐		
B	in	☐		
Cin	in	☐		
Sum	out	☐		
Cout	out	☐		
	in	☐		
	in	☐		
	in	☐		

Ports A, B and Cin are set up as inputs, while Sum and Cout are configured as outputs.

Verilog Implementation Code

The 1-bit full adder was written using Verilog's dataflow style, where each output is described through a continuous assign statement rather than procedural logic. This style maps directly onto the Boolean expressions worked out earlier.

```
module full_adder(
    input A,
    input B,
    input Cin,
    output Sum,
    output Cout
);

    assign Sum = A ^ B ^ Cin;
    assign Cout = (A & B) | (B & Cin) | (A & Cin);

endmodule
```

Code Explanation

- module full_adder(...): declares a module called full adder with three inputs (A, B, Cin) and two outputs (Sum, Cout) — this is the reusable block that gets instantiated repeatedly later to build wider adders.
- input A, input B, input Cin: single-bit inputs standing in for the two operand bits and the incoming carry.
- output Sum, output Cout: the two single-bit outputs — the sum bit and the outgoing carry bit.
- assign Sum = A ^ B ^ Cin;: a continuous assignment implementing $SUM = A \oplus B \oplus Cin$ with Verilog's XOR operator (^). Being continuous, Sum updates the instant A, B or Cin changes, just like a real combinational gate network would.
- assign Cout = (A & B) | (B & Cin) | (A & Cin);: implements $Cout = A.B + B.Cin + A.Cin$ with Verilog's AND (&) and OR (|) operators, producing a carry whenever two or more of the three inputs are high.
- endmodule: closes the module definition.

Verilog Testbench Code

A testbench applies all eight possible A, B, Cin combinations to the full adder module in turn and captures the resulting Sum and Cout on the ISim waveform viewer.

```
`timescale 1ns / 1ps

module full_adder_tb;

    // Inputs
    reg A;
    reg B;
    reg Cin;

    // Outputs
    wire Sum;
    wire Cout;

    // Instantiate Full Adder
    full_adder uut (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Sum(Sum),
        .Cout(Cout)
    );

    // Test inputs
    initial begin

        A = 0; B = 0; Cin = 0; #10; // 000
        A = 0; B = 0; Cin = 1; #10; // 001
        A = 0; B = 1; Cin = 0; #10; // 010
        A = 0; B = 1; Cin = 1; #10; // 011
        A = 1; B = 0; Cin = 0; #10; // 100
        A = 1; B = 0; Cin = 1; #10; // 101
        A = 1; B = 1; Cin = 0; #10; // 110
        A = 1; B = 1; Cin = 1; #10; // 111

        $finish;

    end

endmodule
```

Testbench Explanation

- reg A, B, Cin: declared as reg because the testbench drives their values procedurally from inside the initial block — in Verilog, only signals assigned inside a procedural block need the reg type.
- wire Sum, Cout: declared as wire since these outputs come from the full adder module under test (the UUT), not from the testbench.
- full adder uut (...): creates one instance ('uut', unit under test) of full adder and wires its ports to the testbench's own signals via named port mapping (.A(A), .B(B), etc.), so any change to A, B or Cin is picked up immediately.
- initial begin ... end: a procedural block that runs once at time 0 and drives the sequence of test stimuli.
- A = 0; B = 0; Cin = 0; #10;: sets the 000 combination, then waits 10 ns (#10) before moving on, giving the combinational logic time to settle and appear on the waveform.
- The same pattern is repeated for all eight 3-bit combinations (000 through 111), so every row of the truth table gets exercised once.
- \$finish;: ends the simulation once all eight cases have run.

Circuit Diagram / Schematic

Once the design was synthesized (Synthesize - XST), Xilinx ISE generated the RTL schematic for the full adder module. At the top level it shows up as a single black-box symbol with three inputs (A, B, Cin) and two outputs (Sum, Cout), pictured below.

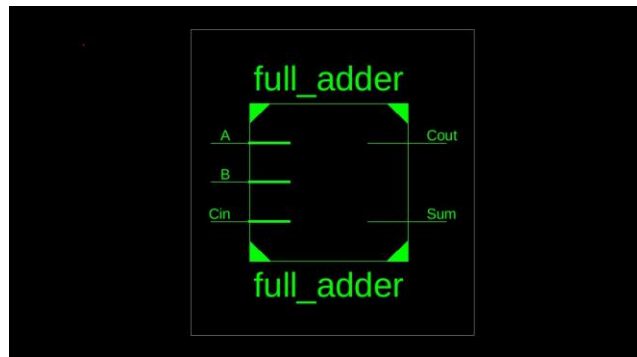


Fig 1: full adder module symbol (top-level black box view)

Opening that symbol exposes the gate-level circuit that the XST synthesizer produced from the Verilog source. As the Boolean expressions predict, Sum comes from two cascaded XOR gates (the 3-input XOR $A \oplus B \oplus Cin$), while Cout comes from two AND gates feeding a final OR gate ($A.B + B.Cin + A.Cin$).

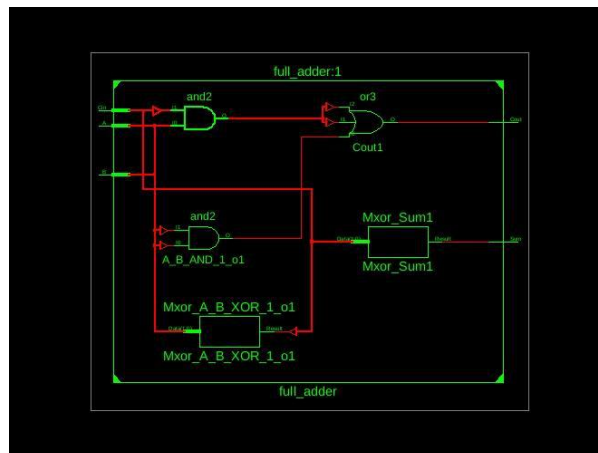


Fig 2: RTL (gate-level) schematic of the designed 1-bit full adder generated using Xilinx ISE

Expected Output (Truth Table)

Combining the Boolean expressions with the hand calculations above gives the following truth table of expected Sum and Cout values for every possible combination of the three inputs.

A	B	Cin	Expected Sum	Expected Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

This matches the theory: Sum reads HIGH exactly when an odd number of the three inputs are HIGH, and Cout reads HIGH exactly when two or more of them are HIGH — both patterns are visible directly in the table.

Simulation and Result

The 1-bit full adder was run through ISim for 80 ns, applying all eight input combinations 10 ns apart, in the same order as the truth table above. The waveform captures below show the signal values at each 10 ns checkpoint, with the Name/Value panel on the left confirming the exact levels at that moment.

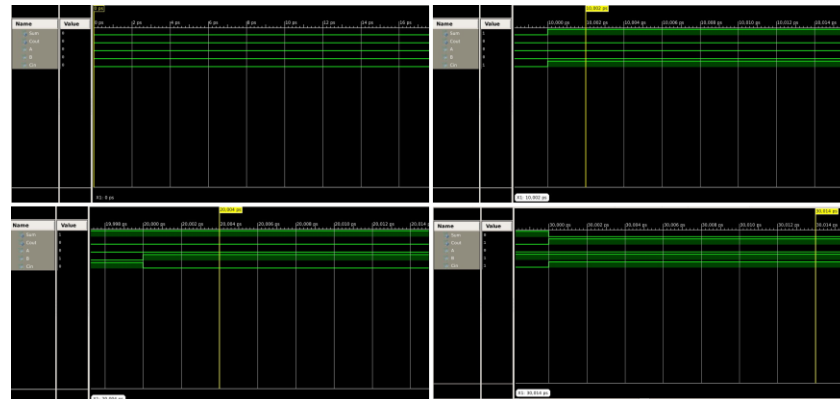


Fig 3 (a): ISim waveforms for Cases 1-4 of the 1-bit full adder

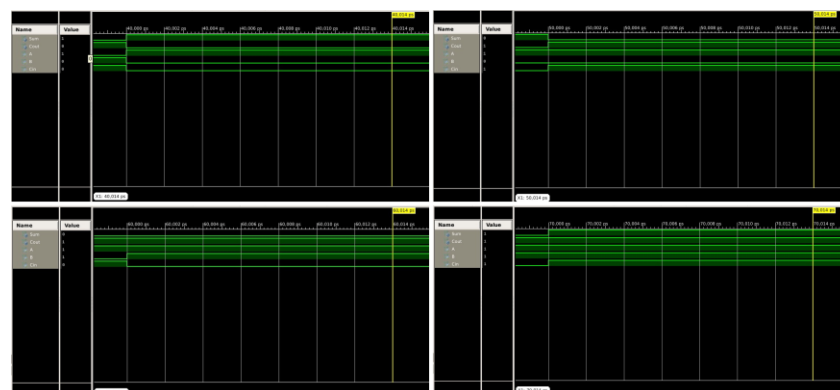


Fig 3 (b): ISim waveforms for Cases 5-8 of the 1-bit full adder

Simulation Output Table

A	B	Cin	Simulated Sum	Simulated Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Result Analysis

Lining up the Simulation Output Table against the Expected Output truth table shows that the simulated Sum and Cout agree with the hand-calculated values for every one of the eight input combinations, with no mismatches. From this it follows that:

- The dataflow Verilog description (assign Sum = A ^ B ^ Cin; and assign Cout = (A & B) | (B & Cin) | (A & Cin);) correctly implements the Boolean expressions of a full adder.
- The XST synthesizer translated the Verilog into the expected gate-level structure — two XOR gates plus two AND gates and an OR gate — exactly as seen in the RTL schematic.
- The circuit reacts correctly and combinationally to every change in A, B and Cin, with no stray 'unknown' (X/U) states showing up once the design was corrected.

The simulation therefore confirms that this 1-bit full adder performs binary addition correctly across every possible input combination, and that the design is ready to serve as the core building block for a wider ripple-carry adder, which is exactly what Experiment 03 (the 8-bit full adder) builds on.

GitHub Repository

[The completed source code](#)

Conclusion

This experiment gave hands-on practice in designing and simulating a 1-bit full adder with Verilog HDL inside Xilinx ISE. It walked through deriving the Boolean expressions from first principles, hand-calculating the expected outputs, writing dataflow-style Verilog, generating the RTL schematic, building a testbench, and running a behavioral simulation with ISim. The simulated Sum and Cout results for every one of the eight input combinations lined up completely with the hand-derived truth table, confirming that the full adder works correctly and that every objective of the experiment was met. The verified full adder module also went on to serve as the essential building block that was reused eight times, in a ripple-carry arrangement, to construct the 8-bit full adder covered in Experiment 03.